

ch14 Indexing

索引用来加速数据的获取

Basic Concepts

search key: 用来查找记录的属性

Index File 存放一系列的 **Index Entries**: [搜索键的值->对应数据的位置]

两类基本索引: **Ordered index, Hash index**

评价索引是否好的标准:

- 访问类型: 索引支持精确查询还是范围查询
- 访问时间
- 插入、删除时间
- 空间开销

Ordered Indices

Ordered Index: 索引按搜索键排好序

Clustering/Primary index: 这个索引key顺序和文件本身的排序一致, 那么这个索引就是~

Non-clustering/Secondary index: 这个索引的顺序和文件排序不一样, 这个索引就是非聚类

Index-sequential file: 文件按照某个搜索值排序, 并且这个搜索值上有**聚类索引**

Dense Index Files

每个搜索键值都在文件里出现

Sparse Index Files

只给文件中一部分的搜索值建立索引, 省空间但是速度慢, 必须要求文件有序

比较好的做法是把每个block建一个索引, 然后指向这个block的最小搜索值

Secondary Indices

必须用dense index, 同时每个索引先指向一个bucket, 因为一个索引值可能对应多个元组, 要把对应元组的位置集中放在一个bucket里面

dense index+bucket

聚类索引适合范围查询和顺序扫描; 非聚类索引适合精确查找, 但大量顺序读取时可能很慢。

Muti-level Index 多级索引

当索引太大, 内存放不下时, 再给索引本身建一个稀疏索引

Index Update

删除

对dense index:删掉这个记录同时索引的指针也要删掉

对sparse index:如果这个值在索引里面存在就用下一条的key替代，如果没有就直接删

插入

Dense: 如果不在索引中就插入一个新索引

Sparse: 通常不用改索引，除非产生了新的block

Indices on Multiple Keys

多个属性一起建索引，比如 (name, ID)

排序的时候先比其中一个属性 name，再比较另一个 ID

适合查名字，或者名字和ID同时查，但是不适合只查ID

B+ Tree Index Files

所有叶节点的深度一样

内部节点的孩子数在 $\lceil n/2 \rceil$ 到 n

叶节点的key值在 $\lceil (n-1)/2 \rceil$ 到 $n-1$

根至少有两个孩子，如果它是唯一的节点那么可以有0-n-1个key

Node structure

$P_1 \mid K_1 \mid P_2 \mid K_2 \mid \dots \mid P_{n-1} \mid K_{n-1} \mid P_n$

K是搜索键的值，P是指针：在非叶子节点里指向子节点，在叶节点里指向bucket

升序排列

叶节点：每个叶节点之间用指针连接

非叶节点：指向下面节点

Queries

精确查询 find(V):从根节点开始，根据key逐步判断最后在叶节点里面找到

范围查询：先找左边的，然后再叶节点链表遍历到右边的停止

查询很快

插入

删除

先从叶节点删除，如果叶节点的key还在范围内就结束

如果不在范围内，先考虑合并节点，如果不能合并，就向旁边的节点拿一个

B Tree Index Files

每个搜索值只出现一次，所以非叶子节点不仅要指向子节点，还要能直接指向真实记录。

Hashing

静态哈希

用哈希函数直接决定数据放在哪个 bucket

Bucket:放多条记录或索引项，**哈希函数**用来把搜索键值映射到bucket

Collision: 不同 key 可能被映射到同一个 bucket，在bucket里面还要顺序查找

Hash index 桶里面存索引项，**Hash file organization** 桶里面存真实的记录

Bucket Overflow:桶太少或者数据倾斜造成，如果该桶溢出，就在后面接一个overflow bucket, bucket 1 → overflow bucket → overflow bucket 为溢出链 **overflow chaining**

静态哈希的缺点是桶的数量已经固定，数据库大小变化时不灵活

动态哈希

Periodic rehashing 定期重哈希: 直接建一个更大的哈希表，把所有的数据重新哈希一遍

Linear Hashing 线性哈希: 每次只分裂一部分的桶

Extendable Hashing 可扩展哈希: 用一个目录指向桶，目录可以变大，桶不一定全部翻倍

索引的语法

```
1 create index index_name on table_name(attribute_list);
2 drop index index_name;
```

ch24 Advanced Indexing

Bitmap Index

就是给每个属性值做一串 0/1 标记；查询多个条件时，用 AND/OR/NOT 快速算出哪些记录满足条件

Spatial Data

空间数据就是带位置区域信息的数据

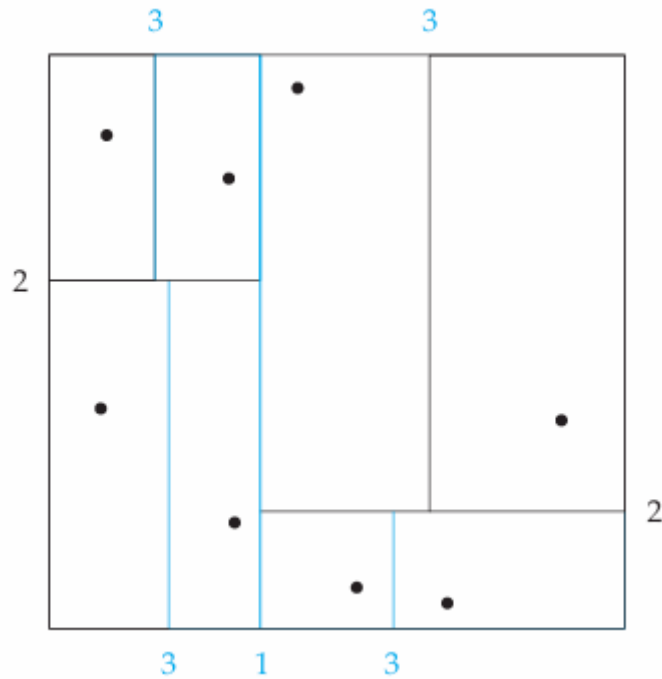
常见查询：

- Nearest neighbor queries:找离最近的餐厅
- Range queries: 找某个区域的点
- 区域的交或并
- Spatial Join: 根据位置关系连接两张表

K-D Tree

比如在二维平面的点，第一次沿水平方向平分，第二次在这两个块分别沿竖直方向，第三次再沿水平方向把四个分成八个...

每次划分保证两侧点数量尽量相等，直到有一个节点少于要求的点的数量



做矩形范围查询：找落在矩形区域的所有点，这时可以根据k-d树划分线的坐标进行筛选，只进入左子树还是右子树，还是都要查。

Quadtrees

每次把当前区域平均分成四块，对点数多的区域继续分块，直到叶节点中的点数不超过要求，此时每个树节点的子节点数量都是4

R Tree

用来存储二维数据

父节点用一个大矩形**bounding box**包住子对象，查询时只进入和目标区域相交的矩形；但因为矩形可能重叠，所以有时要查多个分支

Temporal Data

不止保存内容，还保存它从什么时候到什么时候有效 `start time → end time`

如果结束时间是 `9999-12-31`，说明这个记录从开始到现在都有效，结束时间未知

查询：

```
1 where start <= '2005-01-01' and end > '2005-01-01'
2 或者
3 where time interval overlaps [t1, t2]
```

建索引： `(course_id, start, end)`，对于end无限大的，可以单独建一个索引 `(a, start-time)`

ch15 Query Processing

如何测量查询代价，评估关系代数操作的算法，如何如何结合算法去评估完整的操作

Basic Steps in Query Processing

Parsing and translation 解析和翻译

检查语法，确认关系，然后翻译成内部形式：关系代数

Optimization

关系代数表达式 = 逻辑上要做什么，执行计划 = 物理上怎么执行

在所有结果相同的执行计划中，选择预计代价最低的执行计划

Evaluation

执行计划，访问数据，返回结果

Measures of Query Cost

代价主要来自：**磁盘disk access I/O**，CPU，network communication

衡量代价的方式：

- response time: 用户等的时间
- total resource consumption: 总资源消耗

这里用**总资源消耗**作为指标，同时忽略CPU代价(并行系统必须考虑网络交流)

disk cost 主要包括：seek寻道，read blocks，write blocks

cost 简化公式：用两个指标：读写数据块的个数，寻道的次数

设 t_T =number of transfers, t_S =number of seeks

$$b * t_T + S * t_S$$

buffer 对真实执行时间影响很大，但很难提前精确估计，同一个算法，在内存多和内存少的情况下，代价可能不同

Selection Operation

File scan

A1:linear search

从文件的第一个块开始读，一个接一个

$cost = b_r + 1seek$, b_r 是r占的数据块的数量

如果select条件是key属性，那么找到那条记录就可以停下， $cost = b_r / 2$ 取平均值

Index Scan

WHERE 条件中的属性必须是索引的 search key

clustering index: 数据按照该索引排序存放

A2 (clustering index, equality on candidate key) 获取一条记录

equality on candidate key: 候选键的等值搜索

$$Cost = (h_i + 1) * (t_T + t_S)$$

用B+树搜索, h_i 是树的高度, 1是获取数据, t_T 和 t_S 分别是传输一次block和一次seek用时, 访问一次block就需要 $t_T + t_S$ 所以用乘法

A3 (clustering index, equality on non-key) 获取多条记录

记录大概率连续存放

$$Cost = h_i * (t_T + t_S) + t_S + t_T * b$$

先根据B+树进行 h_i 次搜索, 定位到叶节点, 然后用 t_S seek到第一个block,之后对b个连续的block读取

A4(secondary index 无排序, equality on key/non-key)

On key:

$$Cost = (h_i + 1) * (t_T + t_S), \text{与A2类似}$$

On non-key:

$$Cost = (h_i + n) * (t_T + t_S)$$

very expensive

A5(clustering index, comparison)

comparison: `where salary >= 8000` (前面的是等值查询, 现在是范围查询)

$A \geq v$:

查到第一个满足条件的数据后向下顺序查询, 代价和A3类似

$$Cost = h_i * (t_T + t_S) + t_S + t_T * b$$

$A < v$:

可以直接从头扫描直到遇到第一个不符合条件的数据

A6(non-clustering index, comparison)

$A \geq v$:

先在索引中找到第一个满足的索引项, 然后沿着索引叶子节点继续扫描, 找到所有满足条件的 record pointers, 再根据这些 pointers 去取真正的数据记录

$A \leq v$:

从开头扫描, 直到遇到第一个 $A > v$ 的索引

代价和A4相同

Conjunction

`where ...and...and...`

A7(conjunctive selection using one index)

用其中一个代价最低的条件使用索引取到内存, 再判断其他条件, 其中这个代价来自A1~A6中的公式

A8(conjunctive selection using composite index)

用复合索引，算法类型(A2,A3,A4)取决于索引的类型

A9(conjunctive selection by intersection of identifiers) 地址的交集

对每个索引得到对应的指针集合，最后求交集得到满足条件的记录

$Cost = \sum (\text{第 } i \text{ 个条件扫描索引的代价}) + \text{根据交集获取记录的代价}$

适合单独每个条件筛出来的数据不少，但多个条件同时满足的数据很少

如果有条件没有索引，那就等取交集之后再检查该条件

Disjunction

where ...or...

A10(disjunctive selection by union of identifiers)

与A9类似，只是取了并集，要求每个条件必须有索引

计算代价方法也和A9类似

Negation

通常用线性扫描，但是如果对应结果少也可以考虑索引

Join Operation

Nested-Loop Join

```
1 SELECT *
2 FROM student as r, takes as s
3 WHERE r.ID = s.ID;
```

对每个r中的元组去s里面遍历，所以是两层的嵌套循环

r是outer relation,s是 inner relation

不需要索引，不要求有序，可用于任何join condition

缺点是要检查每一对tuple

$b_r + n_r * b_s$ block transfers

$n_r = \text{outer的tuple数量}$, $b_r = \text{outer占用的block数量}$, $b_s = \text{inner占用的block数量}$

$n_r + b_r$ seeks

如果较小的表可以整个放进内存，就让它做 inner relation

Block Nested-Loop Join

把外层循环的单位从tuple变成了block

对上面的例子就是

```

1 | 读入 student 的一个 block
2 |     扫描 takes 的所有 block
3 |     比较 student block 里的每个学生
4 |     和 takes block 里的每条选课记录

```

$b_r * b_s + b_r$ **block transfers**

$2 * b_r$ **seeks**

最好的情况: $b_r + b_s$ block transfers + 2 seeks

Indexed Nested-Loop Join

用 inner relation 上的索引, 替代对 inner relation 的全表扫描

必须是等值连接, 而且 inner relation 的对应属性上有索引

$$Cost = b_r * (t_T + t_S) + n_r * c$$

c 是在 inner relation 做一次索引查找的代价, 选择 tuple 少的作为 outer

Sorting

数据太大, 不能一次性放入内存排序, 使用外部归并排序

External Sort-Merge

Create sorted runs

设 M 是内存中可用于排序的 block 数量, $M < \text{relation 的总块数}$, 重复执行: 从磁盘读入 M 个 block, 在内存中对 M 个 block 排序形成 sorted run, 然后写回磁盘

$2b_r$ block transfers

Merge the runs

$N < M$ (N way merge)

内存中 N 个 block 作为 input, 1 个作为 output, 最后把输入缓存清空, 输出一个完整排序

$N \geq M$ (M-1 way merge)

可以同时合并 M-1 个 runs, 因为要留一个输出

merge pass: 从最开始的 runs 到最后的输出经历的阶段

如果每个 run 不只用 1 个 buffer 而是用 b_b 个 buffer, 那么 seek 次数减少, 可以同时合并 $\lceil M/b_b \rceil - 1$ 个 block

Cost Analysis

b_r 是关系 r 中占用 block 的数量, M 是可用内存的块数

- Initial run creation: $2 * b_r$, 读入所有 block, 再写回
- Merge: **每个 merge pass** 的 block transfer 都是 $2 * b_r$, 但是最后的 pass 的写回不算入代价, 最后要 **减去** $1 * b_r$

seeks:

- run: $\lceil b_r / M \rceil * 2$, run 的个数乘二

- merge

Merge-Join

先让两个关系都按照join的属性排序，然后再归并连接，有重复值那么所有的组合都要匹配，适合处理等值条件

cost: $b_r + b_s$ block transfers + $(\lceil b_r/b_b \rceil + \lceil b_s/b_b \rceil)$ seeks + sorting cost if relations are unsorted

b_r, b_s 分别是两个关系的块个数， b_b = 一次连续读写时使用的 buffer block 数量

Hash-Join

适合等值连接和自然连接

对两个表各自进行哈希映射，分别分成n个桶，然后每个对应编号的桶再进行join,较小的关系叫做 build input,较大的关系叫probe input

把小关系的一个分区放进内存，然后建立一个内存中的哈希索引，读取大关系的每个tuple，查找对应的元组

分区数量n要保证每个s_i都能放入内存

Recursive partitioning：（很少需要）如果分区个数n大于M,需要递归分区，先分成M-1个分区

Overflow

skewed: 有的区的元组数量明显多出其他区

hash-table overflow: 某个分区s_i放不进内存，可能是join属性相同或者是哈希函数不好导致的

Resolution: 对s_i再次用新的哈希函数进行分区，同样对r_i也要分区

Evaluation of Expressions

Materialization

自底向上，一步一个操作，中间结果写入磁盘

- 从最底层操作开始，将中间结果**物化（存储）**到临时关系中
 - 然后用这些临时关系作为输入，计算上一层的操作
 - 重复此过程直到顶层

优点：始终适用，实现简单；缺点：IO开销大

总成本 = 各操作成本之和 + 中间结果写入磁盘的成本

Double Buffering: 两个缓冲区，当一个满了之后一边把这个写入磁盘，一边填充另一个缓冲区，为了让磁盘写入和计算并行

Pipelining

一边执行一边向上传递元组，不写磁盘

代价变低，但是不是所有操作都能流水线化，比如排序和哈希连接需要看到全部输入才能输出

Demand-driven

- 系统从顶层操作反复请求下一个元组
- 每个操作根据需求向子操作请求元组

- 操作在两次调用之间必须**维护"状态"**（比如当前扫描到哪了），以便知道下次返回什么

Producer-driven

- 操作主动产生元组并向上推送给父操作
- 父子之间**维护一个缓冲区**：子操作放入，父操作取出
- 缓冲区满了 → 子操作等待，直到有空位再继续产生元组
- 系统调度那些输出缓冲区有空间、且能处理更多输入的操作

ch16

Introduction

evaluation plan：规定了每项操作所使用的算法，以及各项操作的执行如何进行协调

cost-based query optimization 基于代价的查询优化：

1. 利用等价变换规则，生成逻辑等价的查询表达式
2. 给表达式添加执行注解，生成可落地的候选执行计划
3. 根据预估代价，挑选开销最低的计划执行

代价估算依靠：基表的统计元数据，中间结果集的统计推算，各物理算法的成本计算公式

`explain <查询语句>`：展示查询优化其最终的执行计划，附带每步的cost

`explain analyse <query>`：会完整执行这条查询，普通explain不会真实运行SQL

Transformation of Relational Expressions

如果两个关系代数表达式在每个合法的数据库实例上生成的元组集相同，则称这两个表达式是等价的。

Equivalence Rules

$$\sigma_{\theta_1 \wedge \theta_2} = \sigma_{\theta_1}(\sigma_{\theta_2})$$

$$\sigma_{\theta_1}(\sigma_{\theta_2}) = \sigma_{\theta_2}(\sigma_{\theta_1})$$

$$\prod_{L_1}(\prod_{L_2}(\dots(\prod_{L_n}(E))\dots)) = \prod_{L_1}(E)$$

$$\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$$

$$\sigma_{\theta_2}(E_1 \bowtie_{\theta} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$$

$$E_1 \bowtie E_2 = E_2 \bowtie E_1$$

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

当 θ_0 所有属性都只在其中一个表的时候： $\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$

当 θ_1 所有属性只在 E_1 且 θ_2 都在 E_2 中的时候： $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$

当 θ 只包含 $L_1 \cup L_2$ 的属性，那么可以先算投影再根据 θ 连接

集合的**交集和并集**都满足**交换律，结合律**

选择操作 σ 对集合的交，并，差都满足**分配律**

投影操作对集合的并满足**分配律**

Pushing Selections

尽早进行选择 σ 以减少连接 $\bowtie$ 的数量

(一个例子)

Pushing Projects

尽早进行投影 Π 以减少连接 $\bowtie$ 的数量

Join Ordering

如果 $r_2 \bowtie r_3$ 很大，但是 $r_1 \bowtie r_2$ 很小，那可以先算小的

Enumeration of Equivalent Expressions 等价表达式枚举

直接枚举的问题

枚举算法：对当前找到的每一个等价表达式，遍历它内部所有子表达式，对子表达式套用所有使用的等价变换规则，把新的表达式加入集合，直到遍历不产生新的表达式

优化：做剪枝，涉及专属高效枚举逻辑

基于变换的优化实现

空间：共享公共子表达式

通过指针复用完全相同的子查询树，大幅减少内存占用

时间：放弃全量枚举采用动态规划

分步记录小规模表集合的最优连接方案，逐步迭代扩展到多表，避免重复计算相同子集的最优计划

Cost Estimation 代价估计

Catalog Information

- | | |
|---|------------------------------------|
| 1 | nr : 关系 r 的元组数 |
| 2 | lr : 关系 r 中一个元组的大小 |
| 3 | fr : 一个磁盘块能容纳多少个 r 的元组 |
| 4 | br : 存放 r 需要多少个磁盘块 |
| 5 | $v(A, r)$: 关系 r 中属性 A 的不同取值个数 |

Histograms 直方图

- Equi-width histogram: 等宽直方图
- Equi-depth histogram: 等深直方图，划分区间，每个区间内的元组数量近似相等

有些还统计最常见的 n 个值及其出现次数

基于随机取样，信息可能会过期，部分数据库需要手动触发更新

Statistical Information for Cost Estimation

如何估计选择、连接、投影等操作的结果大小

Selection Size Estimation

$\sigma_{A=v}(r)$ 等值选择

满足条件的元组数量约为 $\frac{n_r}{V(A,r)}$ ，如果A是key，那么数量为1

$\sigma_{A \leq v}(r)$ 范围选择

(大于等于是也是对称的)

先边界判断， $v < \min(A, r)$ ，则没有元组满足

否则假设是均匀分布，数量等于总元组乘以区间比例

如果没有最大最小值数据，则估计有一半数据满足

复杂选择运算

选择率 θ_i 是一个元组满足选择条件的概率，如果 n 个元组有 s_i 个元组满足条件，那么选择率为 $\frac{s_i}{n_r}$

$$\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n} : n_r * \frac{s_1 * s_2 * \dots * s_n}{n_r^n}$$

$$\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n} : n_r * (1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \dots * (1 - \frac{s_n}{n_r}))$$

$\sigma_{\theta}(r)$ ：总数减去符合条件的

Joins Size Estimation

如果两个关系R,S没有交集，那么连接等价于笛卡尔积： $n_r * n_s$

如果两个交集是R的主键，那么S的任意一条元组最多匹配R的一条，最后总数不超过S的元组

如果交集是S的外键referencing R，那么连接后的总数就是S的元组数量

如果交集的属性A不是R或S的键：

假设R中每个元组都能匹配，S值均匀分布，那么对R的一个元组，能匹配到S中的 $\frac{n_s}{V(A,S)}$ ，那么一共连接的结果有 $n_r * \frac{n_s}{V(A,S)}$ ；同理，反过来是 $n_r * \frac{n_s}{V(A,R)}$

综上最终的公式取 $n_r * \min(\frac{n_s}{V(A,S)}, \frac{n_s}{V(A,R)})$ ，因为如果一边有很多独特值而另一边没有，对应的大量值根本匹配不上，所以取两个估计中的较小值，可以避免严重高估Join结果大小。

Other Size Estimation

投影： $\Pi_A(r) = V(A, r)$

同一关系的并集或交集：可以直接改写成合并条件做选择

$$|r \cup s| = |r| + |s| (\text{偏大估计})$$

$$|r \cap s| = \min(|r|, |s|)$$

$$|r - s| = |r|$$

V(A,r) 的估计

A中所有属性都来自r, $V(A, R \bowtie S) = \min(V(A, R), n_{R \bowtie S})$

$$V(A, \Pi_A(r)) = V(A, r)$$

嵌套子查询的优化

优化成一次连接会有重复的项

用半连接 semi-join

EXISTS 子查询本质是 Semi-Join, 而不是普通 Join

Materialized Views

把view的查询结果提前算出来, 真正存到数据库里

维护视图:

每次更新后重新计算整个view

Incremental 增量维护

差分 **differential**:只考虑插入和删除

Join 的增量维护

如果view是两张表R, S连接的结果, 当一个表R插入新数据r时, 只需把新数据和另一张表S做连接 $r \bowtie s$, 然后加到原来的view里面就好了

如果删掉了部分数据r,同样把删掉的数据和另一张表连接, 然后在原来的view上删掉

Selection 增量维护

因为选择是对行操作的, 所以只对新增或删除的元组重新用selection然后在对应view中更新就可以

Project 增量维护

投影操作默认去重, 需要额外维护一个**count**: 对每个投影结果记录它由多少个原始元组产生

这样当插入时如果投影结果已经存在就把count+1, 否则加入新元组, count=1

删除时count-1,count=0时才从view删除

集合操作的增量维护

$r \cap s$: 对插入或删除都要检测它是否出处在交集内

$r \cup s$: 插入容易, 删除麻烦(可能另一个表还有这个元组)

查询优化与物化视图的关系

可用view改写查询, 也可以不用view, 直接把view展开

Top-K Queries

```
1 select *
2 from r, s
3 where r.B = s.B
4 order by r.A ascending
5 limit 10;
```

全部做连接然后取前十个太麻烦

优化方法一：一边 join 一边用 index 找匹配

优化方法二：先猜范围，再过滤

Halloween problem

```
1 update R
2 set A = 5 * A
3 where A > 10;
```

把A>10的元组A乘5，这会导致一个满足条件的元组重复被乘

Always defer updates

先扫描满足条件的所有元组，记录旧值新值，最后统一更新

保证不会重复扫描，但是需要额外存储更新列表

Selective defer 按需延迟

只有当改动属性影响where条件时进行延迟

```
1 update R
2 set B = B + 1
3 where A > 10;
```

这样的不需要延迟

连接最小化

查询结果只需要r的属性，join条件是r到s的外键，foreign key 列 r.B 是 not null，没有对 s 的额外过滤条件，s 的 join key 能保证匹配不会产生多倍重复结果，通常 s.B 是 primary key 或 unique key

可以直接去掉连接步骤

Multiquery Optimization 多个查询优化

1. 先分别优化每个 query;
2. 看每个 query 的最优计划里有没有公共子表达式;
3. 如果有，而且共享划算，就共享。

Shared Scans: 扫一遍，同时把数据送给多个查询

Parametric Query Optimization

参数大小会影响优化方案的选择

三个方案：

- 每次运行时重新优化：等 \$1 的值知道了，再优化 query
- Parametric Query Optimization：优化器提前生成一组 plans，每个 plan 对应某些参数范围
- Query Plan Caching：如果优化器认为一个 plan 对大多数参数值都还不错，就缓存它，下次复用。

Adaptive Query Processing: 自适应查询处理

即使 query 开始执行了，数据库也可以边执行边调整计划

运行时选择 operator

运行时监控并重优化

Information Retrieval 信息检索

IR 系统要做的事情，就是从简单的 query 里尽量理解用户背后的真实需求

IR 涉及信息的获取，存储，组织和访问，是从大规模集合中找到满足用户信息需求的**非结构化材料**的过程

IR 系统通常会定义一个 **similarity function**

1. 用户给出信息需求，表现为 **query**
2. 系统中有大量待检索对象，比如 **text or multimedia**
3. 系统计算 query 和每个 object 之间的匹配程度，得到 **relevance score 相关性分数**

IR = query + documents + matching + ranking + feedback

Basic IR concepts and models

IR 一般不处理事务更新，并发控制，恢复，更关心近似搜索，排序

Unstructured data

IR 系统允许对非结构数据进行关键词检索，可包含运算符(and,or),也可涉及语义理解

Semi-structured data

搜索满足以下条件的网页：

- 标题里包含 data
- bullet list 里包含 search

这就不只是普通关键词搜索了，而是利用了网页结构

Assumptions of classical IR 经典IR的基本假设

假设有一个静态的文档集合collection,目标goal是找到相关的信息

Relevance 相关性：Objective客观的，Degree of matching(不是简单的yes/no)，Depend on specific applications依赖具体场景

表示文档和建立索引

Term-document incidence matrix 词项-文档关联矩阵

如果某个文档包含这个词，就填 1；否则填 0

可以支持基本的关键词布尔检索

当数据量很大时会导致矩阵很大，但是同时会很稀疏（取值为1的格子不多），这时候应该只存哪些词在哪些文档中出现过

Inverted index 倒排索引

```
1 Caesar → Doc1, Doc2
2 Brutus → Doc1, Doc2
3 ambitious → Doc2
```

Indexer 工作流程:

把文档拆成token,把每个token绑定他来自哪个文档

合并同一文档的重复词

分成Dictionary: 保存所有出现的term，旁边记录频率和Posting list: 记录出现在哪些文档

```
1 term → document frequency → postings list
```

IR模型

Boolean retrieval model

结果是二元的

布尔模型没有利用词频的信息，不能排序(相关性)

Ranked retrieval model

支持输入自然语言，把最可能有用的文档排在前面

Scoring: 排序检索的基础是打分

不是单独给文档打分，而是针对某个 query 给某个 document 打分

Jaccard 相似：只看词有没有出现，不看词出现了多少次

因此引出 **Term-document count matrix**：记录每个词的词频

定义 $tf_{t,d}$ 为词t在文档d中出现的次数，但是相关性不会随着词频线性增长

Log-frequency weighting

如果词在文档中出现过，那么权重为 $1 + \log_{10}(tf)$ ，如果没出现过，权重为0

对 query 和 document 中共同出现的词，把它们在文档中的权重加起来。

除此之外，在所有文档集合中越少见的词越重要(Stop words 和 rare terms)，因此引出 **idf inverse document frequency**

先定义 **document frequency** df_t ：有多少篇文档包含这个词t，该值越高，说明这个词的信息量越低

定义 **idf**：

$$idf_t = \log_{10}(\frac{N}{df_t})$$

N 是总文档数

Collection frequency 记录的是一个词在整个集合出现多少次， Document frequency 记录有多少篇文档包含这个词。

TF-IDF Weighting

$$tfidf = tf * idf = \log(1 + tf_{t,d}) * \log_{10}(N/df_t)$$

Binary matrix(0/1) → Count matrix(tf) → Weight matrix(tf-idf)

然后现在的文档变成了一个实数向量，每个维度的值对应一个词的TF-IDF权重，我们不会真的存下几千万维的完整向量，而是只存非零项

把用户的查询和文档都变成向量，然后用余弦公式进行相似度计算